NIDS: Evaluation 1 Guide

Everything for the evaluation on 10 October 2026 is in this file. Read sections 1 to 3 first and practise the demo twice. Then read the rest.

This file describes the code exactly as it is today. PROJECT_BRIEF.md describes the full final design, so some things in it (a config file, a worker thread, the SYN flood and ARP rules) are planned and not built yet. If they differ, trust this file.

---

1. What is ready

Part	Status	Where
Synopsis	Done	`synopsis/`
Design diagrams: DFD level 0, DFD level 1, use case, flowchart	Done	`design/Design_Diagrams.pdf`
Live packet capture with counts every second	Working	`sensor/nids.py`
Port scan detection	Working and tested	`sensor/nids.py`
Alert server that saves every alert to a log file	Working	`server/server.js`
Live web dashboard with red alert banner	Working	`server/public/index.html`
Test script that creates a port scan	Working	`sensor/scan_test.py`
SYN flood detection	Not started	Round 2
ARP spoofing detection	Not started	Round 2
Alert history after page refresh, charts	Not started	Round 2
Unit tests, report, PPT	Not started	Later

The whole path from the network card to the browser works, with one of the three attack rules. That is about half of the project.

The code is small on purpose: 61 lines of Python, 25 lines of Node.js, 49 lines of HTML and a 9 line test script.

Already installed on the laptop: Npcap 1.89, Scapy 2.8.0, and the Node packages express and socket.io.

---

2. How to run the demo

You need three PowerShell windows. Normal PowerShell works, because Npcap was installed without the "administrators only" option. If capture ever fails, open PowerShell as Administrator.

Step 1. Start the server (window 1)

```
cd C:\Users\pande\work\nids-project\server
node server.js
```

You will see: `Dashboard running at http://localhost:3000`

Step 2. Open the dashboard

Open Chrome or Edge at `http://localhost:3000`. You see the title, three boxes (TCP, UDP, ARP) showing 0, and an empty Alerts list.

Step 3. Start the sensor (window 2)

```
cd C:\Users\pande\work\nids-project\sensor
python nids.py
```

You will see `NIDS sensor started, press Ctrl+C to stop`, then one line of counts every second. The numbers on the dashboard start changing. Open any website and show that the numbers jump.

Step 4. Attack (window 3)

Place the browser and this window side by side first. The red banner appears less than a second after you press Enter and stays for 5 seconds, while the script itself runs for about 5 seconds. The alert row stays in the list.

```
cd C:\Users\pande\work\nids-project\sensor
python scan_test.py
```

Within a second:

- the dashboard shows a red banner: `ALERT: 23:20:10 Port Scan from 127.0.0.1 to 127.0.0.1 (16 ports)`
- a new row appears in the Alerts list
- window 2 prints a line starting with `ALERT`

Step 5. Show the saved log (window 3)

```
type ..\server\alerts.log
```

Step 6. Stop

Press Ctrl+C in window 2, then in window 1.

If you run `scan_test.py` again within 30 seconds, no new alert appears. That is the cool-down working, not a bug. Wait 30 seconds and run it again.

What to say at each step

- **Step 1:** "This is my alert server in Node.js with Express and Socket.IO. It receives alerts from the sensor, saves them in a log file and pushes them to the browser instantly."
- **Step 3:** "This is my sensor in Python. It uses Scapy with the Npcap driver to capture live packets from my Wi-Fi card and the loopback adapter. Every second it counts TCP, UDP and ARP packets and sends the counts to the server."
- **Step 4:** "Now I act as an attacker. This script knocks on 100 ports of my laptop very fast, the way a port scanner does. My rule saw more than 15 different ports from one IP address within 5 seconds, so it raised an alert. The alert went by HTTP POST to the server, was saved in the log and was pushed to the dashboard through Socket.IO, without refreshing the page."
- **End:** "Capture, counting, the port scan rule and the full alert pipeline are working. Next I will add SYN flood and ARP spoofing detection."

Why the alert says 127.0.0.1

127.0.0.1 is the laptop's own address, called localhost. The test attack comes from my own laptop, so the source and destination are both 127.0.0.1. Windows sends traffic to itself through a special loopback adapter, not through the Wi-Fi card. That is why the sensor listens on two adapters: Wi-Fi for real network traffic, and loopback for this test. A scan from another device on the same Wi-Fi would show that device's IP instead.

If something goes wrong

Problem	Cause	Fix
Window 2 prints `server not reachable`	Server is not running	Do step 1 first
`EADDRINUSE: address already in use :::3000` in window 1	The server is already running in another window	Use the window that is already running, or close it and start again
Dashboard numbers stay at 0	Sensor not running, or wrong adapter name	Check window 2; run `Get-NetAdapter` and put the right name in `INTERFACE` on line 8
Error about interface or Npcap	Npcap missing or no permission	Run PowerShell as Administrator
No alert after `scan_test.py`	Cool-down of 30 seconds	Wait 30 seconds and run it again
Alerts list empty after refreshing the page	Known limitation	The alerts are still in `alerts.log` (step 5)

Before you leave home

1. Run the full demo once in the morning.

2. Take two screenshots with Win+Shift+S: the dashboard with the red banner, and window 2 with the ALERT line. Keep them on your phone as a backup.
3. Charge the laptop.
4. Carry the printed synopsis, the printed `design/Design_Diagrams.pdf` and your logbook.

---

3. The idea in simple words

Think of a security guard at a building. People walk in and out all day, and the guard keeps a count. Normal visitors go to one or two rooms. A thief walks along the corridor trying the handle of every door within a few seconds. The guard notices "one person, many doors, very fast" and sounds the alarm.

My system does the same with network traffic:

- the **packets** are the people
- the **ports** are the doors
- a **port scan** is someone trying many doors quickly to find one that is open
- the **sensor** (`nids.py`) is the guard
- the **dashboard** is the alarm panel

The path of one packet:

Wi-Fi card → Npcap driver → Scapy → `nids.py` (count and check) → HTTP POST → `server.js` (save and forward) → Socket.IO → browser dashboard

---

4. Words you must know

- **Packet:** a small piece of data sent over a network. Every message is split into packets.
- **IP address:** the address of a device, for example 192.168.29.5. 127.0.0.1 always means "this same computer".
- **Port:** a numbered door on a device for one service. Web servers use 443 and 80, SSH uses 22. There are 65,535 ports.
- **TCP:** the protocol for reliable connections such as websites. It starts every connection with a handshake.
- **UDP:** a faster protocol with no handshake. DNS lookups and video calls use it.
- **ARP:** the protocol that finds the MAC address (hardware address) for an IP address on the local network.
- **Three-way handshake:** how a TCP connection starts. The client sends SYN, the server replies SYN-ACK, the client sends ACK.
- **SYN packet:** the first packet of a TCP connection, a knock on a door. A port scanner sends a SYN to each port to see which ones answer.
- **Port scan:** sending SYN packets to many ports quickly to find open services. Attackers do this before an attack, so it is an early warning sign.
- **IDS:** Intrusion Detection System. It watches and warns. It does not block anything. A firewall blocks; an IDS only alerts.
- **NIDS:** a network-based IDS, which watches packets on the network.

- **Npcap:** a Windows driver that lets programs capture raw packets from network cards. Wireshark uses it too.
- **Scapy:** a Python library that captures packets and splits them into layers (IP, TCP, UDP, ARP) that we can read in code.
- **Loopback adapter:** the virtual network card Windows uses for traffic a computer sends to itself.
- **BPF filter:** a short rule such as `tcp or udp or arp` that the driver uses to drop packets we do not need before they reach Python. Wireshark capture filters use the same language.
- **Sliding window:** counting only the events of the last few seconds. Old events fall out of the window as time moves.
- **deque:** a list where adding and removing at both ends is very fast. Used for the sliding window.
- **set:** a collection that keeps only unique values. Used to count different ports.
- **Cool-down:** a waiting time after an alert, so one attacker does not create hundreds of alerts.
- **JSON:** a text format for data, like `{"type": "Port Scan", "src": "127.0.0.1"}`.
- **HTTP POST:** a request that sends data to a server. The sensor uses it to send counts and alerts.
- **Express:** a Node.js library that makes it easy to write web servers and routes.
- **Socket.IO:** a library that keeps a live connection open between the server and the browser, so the server can push new data at any moment. It uses WebSockets.
- **localhost:3000:** my own computer, port 3000, where my server runs.

---

5. `sensor/nids.py` line by line

Lines 1 to 4: Python's own libraries (nothing to install)

- `json` turns a Python dictionary into JSON text for the server.
- `time` gives the current time in seconds. It is used for the 5 second window, the 30 second cool-down and the 1 second timer.
- `urllib.request` sends HTTP requests to the server, like a browser does.
- `collections` gives `defaultdict` and `deque`.

Line 6: from Scapy, import the layer types `ARP`, `IP`, `TCP`, `UDP` (to check what a packet contains), `conf` (Scapy settings, used for the loopback adapter name) and `sniff` (the function that captures packets).

Lines 8 to 12: settings. Names in capital letters are constants, so they are easy to find and change.

- `INTERFACE = "Wi-Fi"`: the name of the network card as Windows shows it.
- `SERVER`: the address of my Node.js server.
- `PORT_LIMIT = 15` and `WINDOW = 5`: alert when one IP touches more than 15 ports within 5 seconds.
- `COOLDOWN = 30`: after an alert, stay quiet about the same IP for 30 seconds.

Lines 14 to 17: memory of the program

- `counts`: a dictionary with the number of TCP, UDP and ARP packets in the current second.

- `syn_history`: for each source IP, a deque of `(time, port)` pairs. `defaultdict(deque)` creates an empty deque automatically the first time a new IP appears, so there is no KeyError.
- `last_alert`: the time of the last alert for each IP, used for the cool-down.
- `last_stats`: the time when counts were last sent.

Lines 20 to 25: `send(path, data)` sends data to the server

- Line 21 builds the request: the full address (`SERVER + path`, for example `http://localhost:3000/alert`), the body (the dictionary turned into JSON text, then into bytes with `.encode()`), and a header that says the body is JSON. A request that carries data is sent as POST.
- Line 23 sends it and waits at most 1 second.
- Lines 22, 24, 25: `try` and `except`. If the server is off, `urlopen` raises an error. Instead of crashing, the program prints `server not reachable` and keeps capturing.

Lines 28 to 42: `check_port_scan(packet)`, the detection rule

- Line 29: continue only if the packet has an IP layer, a TCP layer, and its TCP flags are exactly `"S"`, meaning only the SYN bit is on. A reply such as SYN-ACK (`"SA"`) is ignored. If the condition fails, `return` leaves the function early.
- Line 31: `src` is the sender's IP address.
- Line 32: `now` is the current time in seconds.
- Line 33: `history` is this IP's deque. A new IP gets an empty one.
- Line 34: add `(time, destination port)` at the right end.
- Lines 35 and 36: the sliding window. `history[0]` is the oldest entry and `history[0][0]` is its time. While the oldest entry is more than 5 seconds old, remove it from the left with `popleft()`. Now the deque holds only the last 5 seconds. The loop always stops before the deque is empty, because the packet we just added is 0 seconds old.
- Line 37: take only the ports and put them in a set. A set keeps unique values, so 50 SYNs to port 443 count as one port. The _ means "ignore the time here".
- Line 38: two conditions must both be true: more than 15 different ports, and more than 30 seconds since the last alert for this IP. `last_alert.get(src, 0)` gives 0 if this IP never caused an alert, so the second condition is true the first time.
- Line 39: remember the alert time for the cool-down.
- Line 40: build the alert: type, source IP, destination IP, number of ports, and the clock time as hours:minutes:seconds.
- Line 41: print it in the terminal.
- Line 42: send it to the server at `/alert`.

Lines 45 to 56: `handle(packet)`, which Scapy calls for every captured packet

- Line 46: `global last_stats` lets the function change the `last_stats` variable defined outside it. Without `global`, Python would make a new local variable. `counts` does not need `global` because we change values inside the dictionary, not the dictionary itself.
- Lines 47 to 49: go through three pairs of (name, layer). If the packet has that layer, add 1 to its count.
- Line 50: run the port scan rule on this packet.
- Line 51: has at least 1 second passed since counts were last sent?
- Lines 52 and 53: print the counts and send them to the server at `/stats`.
- Lines 54 and 55: reset every count to 0 for the next second.

- Line 56: remember the time of this send.

Lines 59 to 61: start capturing

- Line 59: the code below runs only when this file is run directly with `python nids.py`, not when another file imports it. That will help when writing tests.
- Line 61: `sniff` captures packets until Ctrl+C:
  - `iface=[INTERFACE, conf.loopback_name]`: listen on the Wi-Fi card and on the Npcap loopback adapter.
  - `filter="(tcp or udp or arp) and not port 3000"`: a BPF filter applied inside the driver. Python receives only TCP, UDP and ARP packets. `not port 3000` ignores the sensor's own messages to the dashboard server, otherwise the system would count its own traffic.
  - `prn=handle`: call `handle` for each packet.
  - `store=False`: do not keep packets in memory, otherwise memory would grow forever.

---

6. `server/server.js` line by line

- **Line 1:** `express`, a library that makes web servers and routes simple.
- **Line 2:** `http`, Node's built-in HTTP server. Socket.IO needs an HTTP server to attach to.
- **Line 3:** `fs`, Node's built-in file system module, used to write `alerts.log`.
- **Line 4:** the `Server` class from socket.io. The curly braces take just `Server` out of the library.
- **Line 6:** create the Express app.
- **Line 7:** create an HTTP server that uses the app for requests.
- **Line 8:** create the Socket.IO server on the same HTTP server, so port 3000 serves both the web page and the live connection.
- **Line 10:** `express.json()` reads JSON request bodies and puts them in `req.body`.
- **Line 11:** serve the files in the `public` folder, so `index.html` opens at `http://localhost:3000`. `__dirname` is the folder of `server.js`, so this works from any terminal folder.
- **Lines 13 to 16:** route `POST /stats`. When the sensor sends counts, `io.emit("stats", ...)` forwards them to every open browser, then the server replies 200 OK.
- **Lines 18 to 23:** route `POST /alert`.
  - Line 19 checks the alert. If `type` or `src` is missing, the server replies 400 Bad Request and stops, so broken data is never logged.
  - Line 20 adds the alert as one line of JSON to `alerts.log`, a permanent record.
  - Line 21 pushes the alert to every open browser.
  - Line 22 replies 200 OK.
- **Line 25:** start listening on port 3000 and print the address.

---

7. `server/public/index.html` explained

- The `<style>` block is the CSS: white boxes for the counts, a red banner and red-edged alert rows.
- Line 16: the banner `div` starts `hidden`.

- Lines 20 to 22: three `span` elements with the ids `TCP`, `UDP` and `ARP`. These ids are the same as the keys of the `counts` dictionary in Python, so the JavaScript can find each box by name.
- Line 28: Socket.IO's server automatically gives the browser its client library at `/socket.io/socket.io.js`.
- Line 30: `io()` connects to the server that sent the page.
- Lines 32 to 34: every time a `stats` event arrives, loop over its keys and put each number into the box with the same id.
- Lines 36 to 46: every time an `alert` event arrives:
    - build one line of text from the alert
    - create a list item with it and `prepend` it, so the newest alert is on top
    - put the text in the banner and show it
    - line 45: `setTimeout` hides the banner again after 5000 milliseconds, which is 5 seconds
- The code uses `textContent` and not `innerHTML`. Text that came from the network is shown as plain text and can never inject HTML or scripts into the page. That is a security choice.

---

8. `sensor/scan_test.py` explained

- `socket` is Python's own networking library.
- For each port from 1 to 100: create a TCP socket, set a timeout of 0.05 seconds so it does not wait, and call `connect_ex`. That makes Windows send a SYN packet to that port. `connect_ex` returns an error code instead of crashing when the port is closed. Then close the socket.
- This is what Nmap's "connect scan" does. Use it only on your own machine; that is why it targets 127.0.0.1.

---

9. Why it is built this way

- **Rules instead of machine learning:** every alert can be explained by one counter and one threshold, it needs no training data, and it runs in real time on a laptop.
- **Only SYN packets:** a scanner sends one SYN to each port. Normal browsing opens connections to a few ports (mostly 443 and 80), so it never reaches 15 different ports on one host in 5 seconds.
- **15 ports in 5 seconds:** Nmap's default scan tries 1000 ports in a few seconds, so any real scan crosses this line at once, while normal apps stay far below it. The values are constants, so they can be tuned.
- **Sliding window instead of fixed blocks:** a fixed 5 second block resets to zero, so a scan split across two blocks could be missed. A sliding window always looks at the last 5 seconds.
- **deque and set:** removing the oldest entry from a deque is instant; from a normal list it gets slower as the list grows. A set removes duplicate ports automatically.
- **Cool-down:** without it, one scan of 1000 ports would create hundreds of alerts.
- **Python and Node.js together:** Scapy (Python) is the best tool for packets, and Node.js with Socket.IO is the simplest way to push live data to a browser. HTTP POST with JSON connects them, and either side can be tested alone.

- **Small code:** standard library first (json, time, urllib, collections, socket), only three outside packages (scapy, express, socket.io), and no extra layers. Easy to read, easy to explain, easy to change.

---

10. Questions the panel may ask

What does your project do? It captures live network packets, counts them by protocol, detects port scans with a rule, and shows live counts and red alerts on a web dashboard within a second.

What is an IDS? How is it different from a firewall? An IDS watches traffic and raises alerts. A firewall blocks traffic by rules. My system only watches and warns; it blocks nothing.

What is a port scan and why does it matter? Sending connection requests to many ports quickly to find open services. Attackers scan before they attack, so catching a scan is an early warning.

How does your rule detect it? For each source IP, I keep the SYN packets of the last 5 seconds. If they went to more than 15 different ports, I raise an alert, then wait 30 seconds before alerting on that IP again.

Why only SYN packets? SYN is the first packet of every TCP connection, and a scanner sends one to each port. Replies like SYN-ACK come from the server, so they are not the scanner's knocks.

Why 15 and 5? Normal traffic touches only a few ports on one host, while scanning tools try hundreds of ports per second. The numbers are constants on lines 10 and 11, so they can be tuned after testing.

What is a sliding window? Counting only the last 5 seconds, where old entries are dropped as time passes. Unlike fixed blocks, it never misses a scan that crosses a block boundary.

Why a deque and a set? A deque removes the oldest entry instantly, which a sliding window needs. A set keeps only unique ports.

What is Scapy and what is Npcap? Npcap is the Windows driver that lets a program capture raw packets from the network card. Scapy is the Python library that receives them and splits them into layers I can read.

What is the filter on line 61? A BPF filter that the driver applies before Python sees the packet. It keeps TCP, UDP and ARP, and drops my own dashboard traffic on port 3000.

Why do you capture on the loopback adapter? Windows sends traffic from the laptop to itself through loopback, not through Wi-Fi. Listening there lets me demonstrate an attack on one laptop.

How do Python and Node.js talk? The sensor sends JSON by HTTP POST to `/stats` every second and to `/alert` for each alert.

Why Socket.IO? Why not refresh the page? Socket.IO keeps one connection open, so the server pushes alerts the moment they happen. Refreshing would be slower and would waste requests.

What if the server is off? `send` catches the error, prints `server not reachable`, and capture continues.

Can an attacker avoid your rule? Yes. A very slow scan (fewer than 16 ports every 5 seconds) or a scan spread over many source IPs would stay under the limit. Longer windows and counting per destination would help; that is future work.

What can you see on Wi-Fi? My own laptop's traffic and broadcast traffic such as ARP. Watching a whole network would need a mirror port on a switch.

Is scanning legal? Only on your own devices or with permission. My test script targets my own laptop, 127.0.0.1.

How much is done and what is next? Capture, counting, the port scan rule, the server, the log and the dashboard are done. Next are SYN flood and ARP spoofing detection, alert history, and testing from a second device.

---

11. Changes they may ask you to make live

Practise each one once tonight or in the morning. After changing `nids.py`, stop it with Ctrl+C and run `python nids.py` again.

1. **Make the rule stricter, alert at 11 ports:** line 10, change `PORT_LIMIT = 15` to `PORT_LIMIT = 10`. Run `scan_test.py`; the alert now says 11 ports.
2. **Longer window:** line 11, change `WINDOW = 5` to `WINDOW = 10`.
3. **Shorter cool-down:** line 12, change `COOLDOWN = 30` to `COOLDOWN = 5`. Run `scan_test.py`, wait 6 seconds, run it again: two alerts.
4. **Keep the banner for 10 seconds:** `index.html` line 45, change `5000` to `10000`. Refresh the browser; no restart needed.
5. **Count ping (ICMP) packets too.** This is tested and works:
    - line 6: add `ICMP` to the import, as `from scapy.all import ARP, ICMP, IP, TCP, UDP, conf, sniff`
    - line 14: add `"ICMP": 0` to `counts`
    - line 47: add `("ICMP", ICMP)` to the list of pairs
    - line 61: change the filter to `"(tcp or udp or arp or icmp) and not port 3000"`
    - `index.html`: below line 22, add `<div>ICMP <span id="ICMP">0</span></div>`
    - restart the sensor, refresh the page, run `ping 8.8.8.8` and watch the ICMP box
6. **Move the server to port 4000:** `server.js` line 25, change `3000` to `4000`; `nids.py` line 9, change the address to `http://localhost:4000`; line 61, change `not port 3000` to `not port 4000`. Restart both and open `http://localhost:4000`.

---

12. Known limitations (say them honestly if asked)

1. Counts are sent only when a packet arrives. If the network is completely silent, the dashboard keeps the last numbers. Fix later: a timer thread.

2. Sending to the server happens inside the capture function and can wait up to 1 second. At normal speed this is fine. Fix later: a queue with a separate sender thread.
3. Refreshing the page clears the alert list. The alerts stay in `alerts.log`. Fix later: load history when the page opens.
4. One of the three rules is built so far.
5. On Wi-Fi, the sensor sees only its own laptop's traffic and broadcasts.
6. Slow scans and scans from many IPs can stay under the limit.
7. The server has no login, so any device on the same Wi-Fi could open the dashboard or post a fake alert. Fix later: accept alerts only from the sensor with a shared secret key.

---

13. Logbook entries (up to 10 October)

Date	Work done / issues faced
07 Sep to 13 Sep 2026	Picked my topic: a system that watches network traffic and warns about attacks. Learned what an IDS is.
14 Sep to 20 Sep 2026	Read about Snort, Wireshark and Scapy. Wrote and submitted the synopsis.
21 Sep to 27 Sep 2026	Title approved. Planned the four parts: packet capture, attack rules, server and dashboard.
28 Sep to 04 Oct 2026	Studied TCP, UDP, ARP, the TCP handshake and how port scans work. Issue: Windows needs Npcap to capture packets.
05 Oct to 10 Oct 2026	Installed Npcap and Scapy, made the DFDs and flowchart. Built the packet counter, port scan alert and live dashboard.

Leave the mentor column empty for your guide.

---

14. Plan for evaluation 2 (30 October 2026)

From here you write the code yourself. Bring each piece for review.

Dates	Work
12 Oct to 18 Oct	SYN flood rule: for each destination, count SYN packets in the window and compare with completed handshakes. Alert when SYNs are high and completions stay near zero. Write tests that build packets with Scapy, such as `IP()/TCP(flags="S")`, and pass them to `handle`.
19 Oct to 25 Oct	ARP spoofing rule: at start, learn the gateway's IP and MAC. Alert when an ARP reply claims the gateway IP with a different MAC. Add alert history on page load and severity colours.
26 Oct to 30 Oct	Test from a second device on your own network, measure how fast alerts appear, take screenshots, and start report chapters 3 (design) and 4 (implementation).

The diagrams in `design/` are already numbered as Fig. 3.1 to 3.4, so they can go straight into chapter 3 of the report.